

CS349: Database and Information Systems

Comprehensive Laboratory Technical Report (Labs 1–10)

Roll Numbers: 23B0920, 23B0960 — Term: Spring 2026

1. Introduction

This report provides a concise, structured overview of the software implementations, database schemas, scripts, query optimizations, and data-processing pipelines developed across Labs 1 through 10 in **CS349: Database and Information Systems**. The curriculum spans foundational relational database design, programmatic database connectivity, full-stack MVC web portals, distributed big-data analytics with PySpark, containerization, vector databases with Retrieval-Augmented Generation (RAG), and database query engine profiling.

2. Module-by-Module Technical Summary

2.1 Labs 1 & 2: Relational Schema Design, DDL, and Complex SQL Queries

- **Scope & Schema:** Defined relational DDL specifications enforcing primary, foreign key, and domain constraints across multi-table schemas (University, E-commerce, and Small Relations).
- **SQL Implementation:** Formulated advanced SQL queries involving multi-way inner/outer joins, nested correlated subqueries (`EXISTS`, `IN`), aggregations with `GROUP BY` / `HAVING`, window functions, and set operations (`UNION`, `INTERSECT`, `EXCEPT`).
- **Integrity Enforcement:** Tested transactional inserts, cascading updates/deletions, and referential consistency checks under edge-case data insertions.

2.2 Lab 3: Programmatic Database Access with Python & Psycopg2

- **Architecture:** Built automated Python backend scripts (`23b0920_3a.py`, `23b0920_3b.py`) establishing database sessions using `psycopg2`.
- **Key Features:** Implemented parameterized dynamic query generation preventing SQL injection attacks, transaction rollback handling with `try-except-finally` blocks, cursor management, and automated extraction/formatting of structured relational outputs.

2.3 Lab 4: MVC Web Application with Node.js, Express, EJS, and PostgreSQL

- **System Architecture:** Developed a full-stack University Academic Management Portal based on the Model-View-Controller (MVC) paradigm using Node.js and Express.
- **Backend & Routing (`app.js`):** Implemented asynchronous REST endpoints and session-based authentication separating Student and Instructor roles.
- **Database Layer:** Executed transactional queries for dynamic course registration, prerequisite validation, and grade submission against PostgreSQL.
- **Frontend Views (`.ejs`):** Rendered server-side views for user login, student registration dashboards, and instructor course grading tables.

2.4 Labs 5 & 6: Full-Stack Web Development, REST APIs & State Management

- **Architecture:** Architected end-to-end full-stack web applications featuring decoupled frontend clients communicating with modular backend API servers.
- **API Services:** Built robust CRUD services, JWT/session authentication, middleware request validators, and relational database persistence layers with automated schema migration scripts.

2.5 Lab 7: Distributed Big Data Analytics with Apache PySpark

- **Framework:** Scalable distributed data processing using PySpark RDDs and DataFrames on large-scale dataset logs (`movies.txt`, `transactions.csv`).
- **Tasks & Transformations (`q1.py`, `q2.py`):**
 - Implemented map-reduce workflows, key-value pair transformations (`mapValues`, `reduceByKey`), and distributed joins across partitioned partitions.
 - Executed transaction frequency aggregations, high-value transaction filtering, and ranking queries over massive datasets with minimal memory footprint and parallelized task scheduling.

2.6 Lab 8: Multi-Container Orchestration with Docker Compose

- **Infrastructure:** Orchestrated multi-tier containerized environments using `docker-compose.yml` to establish isolated database networks, application servers, and volume persistence mounts.
- **Environment Isolation:** Configured environment variables, network bridges, automated health checks, and service dependency graphs for reproducible deployments.

2.7 Lab 9: Retrieval-Augmented Generation (RAG) with PostgreSQL & pgvector

- **System Pipeline:** Built an offline RAG system over the IMDb Top 1000 Movies catalog utilizing PostgreSQL, the `pgvector` extension, and a local open-weights LLM (`smollm2:135m`) via the Ollama REST API.

- **Dense Embedding & Storage (model.py):** Ingested movie metadata, generated 576-dimensional dense vector embeddings for plot overviews, and updated database rows using parameterized vector casts (`%s::vector`).
- **Vector Similarity & Indexing (rag.py, experiments.py):** Benchmark-tested vector operators and indexing methods:
 - Distance metrics: Cosine (`<=>`), L2 / Euclidean (`<->`), Inner Product (`<#>`), and Manhattan / L1 (`<+>`).
 - Index structures: Evaluated Sequential Scan vs. HNSW (*Hierarchical Navigable Small World*) vs. IVFFlat (*Inverted File Flat*).
- **Generation Pipeline:** Implemented top- k nearest neighbor retrieval ($k = 5$), constructed dynamic prompt context windows, and synthesized grounded, hallucination-free answers through Ollama’s chat generation API.

2.8 Lab 10: Query Plan Profiling, Indexing Strategies, and Performance Optimization

- **Query Execution Analysis:** Utilized EXPLAIN (ANALYZE, BUFFERS) to inspect cost estimation models, buffer cache hits, recursive Common Table Expressions (CTEs), and execution steps on large transaction tables (e.g., `ipl_deliveries`, `takes`).
- **Access Method Profiling:** Compared Sequential Scans against Index Scans, Bitmap Index Scans, and Index-Only Scans under B-Tree indexes, observing dramatic latency improvements on high-selectivity predicates.

3. Consolidated Summary of System Technologies & Code Artifacts

Lab Module	Core Technologies	Key Code Artifacts	Key Competencies / Concepts Demonstrated
Labs 1 & 2	PostgreSQL, SQL	DDL.sql, ecommerceSchema.sql	Schema design, constraints, nested subqueries, aggregations.
Lab 3	Python, Psycopg2	23b0920_3a.py, 23b0920_3b.py	Programmatic database connectivity, transactions, parameterized queries.
Lab 4	Node.js, Express, EJS, SQL	app.js, views/*.ejs, DDL.sql	MVC architecture, session auth, asynchronous database query pipelines.
Labs 5 & 6	JavaScript, Express, HTML/CSS	backend/, frontend/	Decoupled client-server architecture, RESTful API endpoints, state sync.
Lab 7	Apache PySpark, Python	q1.py, q2.py, movies.txt	Distributed computing, RDD transformations, distributed joins, map-reduce.
Lab 8	Docker, Docker Compose	docker-compose.yml	Container orchestration, service discovery, database persistence mounts.
Lab 9	pgvector, Ollama, Python	model.py, rag.py, experiments.py	Semantic search, vector indexing (HNSW, IVFFlat), RAG with local LLMs.
Lab 10	PostgreSQL Engine, EXPLAIN	DDL.sql, answers.txt	Query plan evaluation, cost estimation, B-tree index scan optimization.

Table 1: Consolidated overview of technologies, source files, and database paradigms implemented across Labs 1–10.

4. Experimental Insights & Cross-Cutting Findings

- **Data Access Layer vs. Engine Performance:** Direct database abstraction via parameterized drivers (`psycopg2`, `pg` in `Node.js`) avoids injection overhead while maximizing throughput via connection reuse.
- **Exact vs. Approximate Search in High Dimensions:** Standard B-Trees fail on high-dimensional vectors, whereas specialized indexes like HNSW in `pgvector` reduce retrieval latency from > 7 ms (sequential scan) down to ≈ 1.1 ms with high recall.
- **Relational Database Profiling:** Understanding execution engine query plans (`EXPLAIN ANALYZE`) is crucial for identifying bottleneck operations (e.g., unindexed filter predicates causing full table scans).
- **Scalability Parallels:** Big data operations scale horizontally across clusters using distributed frameworks like `PySpark`, while containerization (`Docker`) ensures environment parity from local development to production deployment.

5. Conclusion

Across the ten laboratory modules, we designed, implemented, and benchmarked a complete continuum of database and information systems technologies. The implementations progress from fundamental relational schema modeling and query writing to full-stack transactional applications, distributed big-data processing, AI-driven vector retrieval with local LLMs, and database engine performance tuning.